7.6 Case Study: Maintaining Access Frequencies

The positional list ADT is useful in a number of settings. For example, a program that simulates a game of cards could model each person's hand as a positional list (Exercise P-7.47). Since most people keep cards of the same suit together, inserting and removing cards from a person's hand could be implemented using the methods of the positional list ADT, with the positions being determined by a natural order of the suits. Likewise, a simple text editor embeds the notion of positional insertion and deletion, since such editors typically perform all updates relative to a ***cursor***, which represents the current position in the list of characters of text being edited.

In this section, we consider maintaining a collection of elements while keeping track of the number of times each element is accessed. Keeping such access counts allows us to know which elements are among the most popular. Examples of such scenarios include a Web browser that keeps track of a user's most accessed URLs, or a music collection that maintains a list of the most frequently played songs for a user. We model this with a new ***favorites list ADT*** that supports the len and is_empty methods as well as the following:

access(**e**): Access the element e, incrementing its access count, and adding it to the favorites list if it is not already present.

remove(**e**): Remove element e from the favorites list, if present.

top(**k**): Return an iteration of the k most accessed elements.

7.6.1 Using a Sorted List

Our first approach for managing a list of favorites is to store elements in a linked list, keeping them in nonincreasing order of access counts. We access or remove an element by searching the list from the most frequently accessed to the least frequently accessed. Reporting the top k most accessed elements is easy, as they are the first k entries of the list.

To maintain the invariant that elements are stored in nonincreasing order of access counts, we must consider how a single access operation may affect the order. The accessed element's count increases by one, and so it may become larger than one or more of its preceding neighbors in the list, thereby violating the invariant.

Fortunately, we can reestablish the sorted invariant using a technique similar to a single pass of the insertion-sort algorithm, introduced in the previous section. We can perform a backward traversal of the list, starting at the position of the element whose access count has increased, until we locate a valid position after which the element can be relocated.

Using the Composition Pattern

We wish to implement a favorites list by making use of a PositionalList for storage. If elements of the positional list were simply elements of the favorites list, we would be challenged to maintain access counts and to keep the proper count with the associated element as the contents of the list are reordered. We use a general object-oriented design pattern, the ***composition pattern***, in which we define a single object that is composed of two or more other objects. Specifically, we define a nonpublic nested class, _Item, that stores the element and its access count as a single instance. We then maintain our favorites list as a PositionalList of *item* instances, so that the access count for a user's element is embedded alongside it in our representation. (An _Item is never exposed to a user of a FavoritesList.)

```
1  class FavoritesList:
2    """List of elements ordered from most frequently accessed to least."""
3
4    #------------------------------ nested _Item class ------------------------------
5    class _Item:
6      __slots__ = '_value', '_count'              # streamline memory usage
7      def __init__(self, e):
8        self._value = e                           # the user's element
9        self._count = 0                           # access count initially zero
10
11   #------------------------------ nonpublic utilities ------------------------------
12   def _find_position(self, e):
13     """Search for element e and return its Position (or None if not found)."""
14     walk = self._data.first( )
15     while walk is not None and walk.element( )._value != e:
16       walk = self._data.after(walk)
17     return walk
18
19   def _move_up(self, p):
20     """Move item at Position p earlier in the list based on access count."""
21     if p != self._data.first( ):                  # consider moving...
22       cnt = p.element( )._count
23       walk = self._data.before(p)
24       if cnt > walk.element( )._count:           # must shift forward
25         while (walk != self._data.first( ) and
26                cnt > self._data.before(walk).element( )._count):
27           walk = self._data.before(walk)
28         self._data.add_before(walk, self._data.delete(p))     # delete/reinsert
```

Code Fragment 7.18: Class FavoritesList. (Continues in Code Fragment 7.19.)

```
29    #------------------------------ public methods ------------------------------
30    def __init__(self):
31      """Create an empty list of favorites."""
32      self._data = PositionalList( )                # will be list of _Item instances
33
34    def __len__(self):
35      """Return number of entries on favorites list."""
36      return len(self._data)
37
38    def is_empty(self):
39      """Return True if list is empty."""
40      return len(self._data) == 0
41
42    def access(self, e):
43      """Access element e, thereby increasing its access count."""
44      p = self._find_position(e)                    # try to locate existing element
45      if p is None:
46        p = self._data.add_last(self._Item(e))      # if new, place at end
47      p.element( )._count += 1                      # always increment count
48      self._move_up(p)                              # consider moving forward
49
50    def remove(self, e):
51      """Remove element e from the list of favorites."""
52      p = self._find_position(e)                    # try to locate existing element
53      if p is not None:
54        self._data.delete(p)                        # delete, if found
55
56    def top(self, k):
57      """Generate sequence of top k elements in terms of access count."""
58      if not 1 <= k <= len(self):
59        raise ValueError('Illegal value for k')
60      walk = self._data.first( )
61      for j in range(k):
62        item = walk.element( )                      # element of list is _Item
63        yield item._value                           # report user's element
64        walk = self._data.after(walk)
```

Code Fragment 7.19: Class FavoritesList. (Continued from Code Fragment 7.18.)

7.6.2 Using a List with the Move-to-Front Heuristic

The previous implementation of a favorites list performs the access(e) method in time proportional to the index of e in the favorites list. That is, if e is the kth most popular element in the favorites list, then accessing it takes $O(k)$ time. In many real-life access sequences (e.g., Web pages visited by a user), once an element is accessed it is more likely to be accessed again in the near future. Such scenarios are said to possess ***locality of reference***.

A ***heuristic***, or rule of thumb, that attempts to take advantage of the locality of reference that is present in an access sequence is the ***move-to-front heuristic***. To apply this heuristic, each time we access an element we move it all the way to the front of the list. Our hope, of course, is that this element will be accessed again in the near future. Consider, for example, a scenario in which we have n elements and the following series of n^2 accesses:

- element 1 is accessed n times
- element 2 is accessed n times
- $\cdots$
- element n is accessed n times.

If we store the elements sorted by their access counts, inserting each element the first time it is accessed, then

- each access to element 1 runs in $O(1)$ time
- each access to element 2 runs in $O(2)$ time
- $\cdots$
- each access to element n runs in $O(n)$ time.

Thus, the total time for performing the series of accesses is proportional to

$$n + 2n + 3n + \cdots + n \cdot n = n(1 + 2 + 3 + \cdots + n) = n \cdot \frac{n(n+1)}{2},$$

which is $O(n^3)$.

On the other hand, if we use the move-to-front heuristic, inserting each element the first time it is accessed, then

- each subsequent access to element 1 takes $O(1)$ time
- each subsequent access to element 2 takes $O(1)$ time
- $\cdots$
- each subsequent access to element n runs in $O(1)$ time.

So the running time for performing all the accesses in this case is $O(n^2)$. Thus, the move-to-front implementation has faster access times for this scenario. Still, the move-to-front approach is just a heuristic, for there are access sequences where using the move-to-front approach is slower than simply keeping the favorites list ordered by access counts.

The Trade-Offs with the Move-to-Front Heuristic

If we no longer maintain the elements of the favorites list ordered by their access counts, when we are asked to find the k most accessed elements, we need to search for them. We will implement the top(k) method as follows:

1. We copy all entries of our favorites list into another list, named temp.
2. We scan the temp list k times. In each scan, we find the entry with the largest access count, remove this entry from temp, and report it in the results.

This implementation of method top takes $O(kn)$ time. Thus, when k is a constant, method top runs in $O(n)$ time. This occurs, for example, when we want to get the "top ten" list. However, if k is proportional to n, then top runs in $O(n^2)$ time. This occurs, for example, when we want a "top 25%" list.

In Chapter 9 we will introduce a data structure that will allow us to implement top in $O(n + k \log n)$ time (see Exercise P-9.54), and more advanced techniques could be used to perform top in $O(n + k \log k)$ time.

We could easily achieve $O(n \log n)$ time if we use a standard sorting algorithm to reorder the temporary list before reporting the top k (see Chapter 12); this approach would be preferred to the original in the case that k is $\Omega(\log n)$. (Recall the big-Omega notation introduced in Section 3.3.1 to give an asymptotic lower bound on the running time of an algorithm.) There is a more specialized sorting algorithm (see Section 12.4.2) that can take advantage of the fact that access counts are integers in order to achieve $O(n)$ time for top, for any value of k.

Implementing the Move-to-Front Heuristic in Python

We give an implementation of a favorites list using the move-to-front heuristic in Code Fragment 7.20. The new FavoritesListMTF class inherits most of its functionality from the original FavoritesList as a base class.

By our original design, the access method of the original class relies on a non-public utility named _move_up to enact the potential shifting of an element forward in the list, after its access count had been incremented. Therefore, we implement the move-to-front heuristic by simply overriding the _move_up method so that each accessed element is moved directly to the front of the list (if not already there). This action is easily implemented by means of the positional list ADT.

The more complex portion of our FavoritesListMTF class is the new definition for the top method. We rely on the first of the approaches outlined above, inserting copies of the items into a temporary list and then repeatedly finding, reporting, and removing an element that has the largest access count of those remaining.

```
class FavoritesListMTF(FavoritesList):
  """List of elements ordered with move-to-front heuristic."""

  # we override _move_up to provide move-to-front semantics
  def _move_up(self, p):
    """Move accessed item at Position p to front of list."""
    if p != self._data.first( ):
      self._data.add_first(self._data.delete(p))        # delete/reinsert

  # we override top because list is no longer sorted
  def top(self, k):
    """Generate sequence of top k elements in terms of access count."""
    if not 1 <= k <= len(self):
      raise ValueError('Illegal value for k')

    # we begin by making a copy of the original list
    temp = PositionalList( )
    for item in self._data:                  # positional lists support iteration
      temp.add_last(item)

    # we repeatedly find, report, and remove element with largest count
    for j in range(k):
      # find and report next highest from temp
      highPos = temp.first( )
      walk = temp.after(highPos)
      while walk is not None:
        if walk.element( )._count > highPos.element( )._count:
          highPos = walk
        walk = temp.after(walk)
      # we have found the element with highest count
      yield highPos.element( )._value                # report element to user
      temp.delete(highPos)                           # remove from temp list
```

Code Fragment 7.20: Class FavoritesListMTF implementing the move-to-front heuristic. This class extends FavoritesList (Code Fragments 7.18 and 7.19) and overrides methods _move_up and top.